SAVEETHA
INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Programme: **B.E / B.Tech**

Course Code: **CSA17**

Course Name: **ARTIFICIAL INTELLIGENCE**

Submission Date: 16-11-2026

Date of Issue : 5-11-2026 Max Marks: **50 Marks**

ASSIGNMENT 3

Course Outcomes

On successful completion of the course, students will be able to:

CO	Course Outcome Statement
CO3	Analyze knowledge representation and reasoning using propositional logic, FOL, inference, resolution, chaining methods, knowledge representation to perform logical reasoning for drawing valid conclusions and decision making.

Evidence Rule Common to All Assignments

Every implemented task must be supported by: **Source Code → Test Input → Expected Behaviour → Actual Output → Performance Metrics (table/graph over at least three input sizes or seeds) → Interpretation.** A screenshot showing only a final answer is not sufficient. Operation counts and task metrics are primary evidence; execution time is secondary. Wherever a task says Predict, the prediction must be written before the experiment is run. Programming language: Python 3 (Jupyter / Colab).

ASSIGNMENT 3 – CO3
ARTIFICIAL INTELLIGENCE – CSA17

CO3 Coverage: Logical agents – Propositional logic – Reasoning patterns in propositional logic – Resolution – First-order logic – Knowledge engineering in FOL – Inference in first-order logic

– Propositional versus first-order inference – Unification and lifting – Forward chaining – Backward chaining – Resolution.

Mode: Pair Work – 2 Students per Group
Duration: 1 Week **Total Marks:** 50

Submission Date: _____

Pair-Work Principle

Both students are responsible for understanding the complete assignment. Each task contains clearly identified individual responsibilities so that the contribution and understanding of each student can be independently verified.

Student A and Student B must **not** divide the assignment task-wise. Both students must participate in every task. The individual responsibilities alternate across tasks so that both students show evidence of: **Design → Implementation → Analysis → Evaluation**.

TASK 1 – Logical Agents: Knowledge Bases, Entailment and Reasoning Patterns (6 Marks)

Part A – Knowledge Base and Model Checking in the Wumpus World

Shared Group Work: Together encode the rules of an n × n Wumpus world in propositional logic (pit, wumpus, breeze and stench symbols for every square and the rules relating percepts to neighbouring hazards). Implement a truth-table entailment checker and query it with percept sequences that you design, for example, “Is square [2,2] safe?”.

Student A – Design and Mathematical Analysis

- Count the proposition symbols and sentences as a function of n
- Derive the number of models 2^m and identify the minimal set of symbols relevant to one query
- Explain what soundness and completeness mean for model checking

Student B – Experimental Validation

- Instrument the checker to record models enumerated, KB evaluations and time
- Run it for n = 2, 3, 4 (state where the full symbol set becomes infeasible) and again with relevant-symbol restriction
- Include one entailed query, one query whose negation is entailed and one query for which neither is entailed

Part B – Reasoning Patterns

Shared Group Work: Using the same Wumpus-world KB, derive the safety of at least two squares by applying inference rules (Modus Ponens, And-Elimination, Unit Resolution) instead of model checking.

Student A – Experimental Investigation: Implement the inference-rule derivation, record the number of inference steps and compare with the model-checking cost for the same query.

Student B – Soundness Reasoning: Prove the soundness of each rule used with a truth table and explain why a derived sentence must hold in every model of the KB, and why "not derived" is not the same as "false".

Cross-Understanding Requirement: Each student must be able to explain the other student's component.

Evidence Expected: KB listing, symbol-count derivation, checker code and outputs for the three query types, inferencerule derivations with step counts, soundness tables and a comparison of the two reasoning approaches.

TASK 2 – The Quality of Reasoning Method: Model Checking versus Resolution (7 Marks)

Shared Group Work: Implement both a truth-table model checker and a propositional resolution prover (CNF conversion followed by resolution refutation). Design a smart-building access-control policy of at least 10 rules (badge validity, time of day, visitor status, emergency override) that contains implication, biconditional and disjunction.

The group must design test knowledge bases that make each method look strong or weak. Do not use a single sample KB.

Student A – Resolution Investigation

- Convert three sentences to CNF by hand and construct a resolution proof for one entailed query
- Record clauses generated, resolvents attempted, tautologies discarded, subsumed clauses removed and time as the KB grows (implication chains of length 10, 20, 40, 80)
- Test a query that is not entailed and explain why the process terminates

Student B – Model-Checking Comparative Analysis

- Record models enumerated and time for the same KBs and queries
- Identify a KB family on which model checking is faster than resolution and a family on which it is far slower, and explain what structural property causes the difference

Joint Design Question

If you were designing a reasoning component whose KB size and query types are not known in advance, what information would you want before deciding between these two methods? Support your answer with your own measurements.

Evidence Expected: Manual CNF conversion, proof tree, both implementations, clause and model counts across KB sizes, designed KB families and the joint design discussion.

TASK 3 – Breaking the Expressiveness Barrier: Knowledge Engineering in First-Order Logic (7 Marks)

Problem Context – University Exam-Eligibility Advisor

Regulations decide whether a student may write an examination (minimum attendance, prerequisites cleared, fees paid, disciplinary status, approved leave).

Shared Group Work: Follow the knowledge-engineering process (identify the task, assemble knowledge, decide vocabulary, encode general knowledge, encode the instance, pose queries, debug the KB). Build two encodings of the same regulations: a **propositional** one (one set of rules per student and course) and a **first-order** one (general axioms plus facts).

Student A – Design Derivation

- Define the FOL vocabulary and write at least 12 axioms with quantifiers, including one existential statement
- Derive how the number of sentences in the propositional encoding grows with students s and courses c, and show how the FOL axioms remain constant

Student B – Crossover Investigation

- Design at least 15 competency queries (5 that succeed, 5 that fail, 5 returning bindings) and document at least three KB defects found and fixed
- Measure KB size and inference time for both encodings as s and c grow, and locate the range of sizes at which the FOL encoding becomes preferable despite unification overhead

Joint Conclusion

Why does greater expressiveness not automatically give better reasoning performance for every KB size? Support your answer with your measurements.

Evidence Expected: Vocabulary and axioms, growth derivation, query suite with expected and actual results, defect log, size/time tables and a reasoned conclusion.

TASK 4 – The Reality of Unification and Lifted

Inference (7 Marks) The responsibilities in this task are

reversed compared with Task 3.

Shared Group Work: Implement UNIFY (with substitution composition and an optional occurs check), lifted Modus Ponens, and propositionalisation of a small FOL KB (instantiating universal sentences with all ground terms).

Student B – Logical Design and Correctness

- Derive the most general unifier by hand for five pairs of terms that you design, including a pair where the occurs check matters and one where standardising apart is essential

- Explain why lifted inference is sound and why it can avoid generating ground instances

Student A – Experimental and Engineering Analysis

- Measure the number of ground clauses as the number of constants grows (k = 3 to 30) and as predicate arity grows
- Introduce a function symbol and show how the number of ground terms grows with the term-depth bound
- Compare inference steps and time for lifted versus propositional inference on the same query

Joint Engineering Decision

Under what conditions would you propositionalise a knowledge base despite the availability of lifted inference? State the conditions using your own evidence.

Evidence Expected: Unifier code and tests, hand derivations, ground-clause growth tables and plots, a lifted-versuspropositional comparison and the engineering decision.

TASK 5 – Upgrading Resolution from Propositional to First-Order Logic (6 Marks)

In Task 2, propositional resolution was applied to a policy KB. Review that solution before attempting this task.

Shared Group Work: Using the following English statements, design a first-order resolution prover:

1. Every employee who handled a restricted document is authorised.
2. Every document containing patient data is restricted.
3. Some employee handled every document that was audited.
4. Report R1 contains patient data and R1 was audited.

Convert the statements to FOL and CNF (with Skolemisation), implement resolution using your Task 4 unifier, and prove that an authorised employee exists.

Student A – Conversion and Proof Design: Show every conversion step, justify the Skolem function or constant, draw the proof tree and implement one resolution strategy of your choice (unit preference, set-of-support or input resolution).

Student B – Comparative Evidence: Record clauses generated, resolution steps, proof length and unifications with and without the strategy. Pose a second query that is **not** entailed and report the behaviour of your prover under a step cap. Compare with the propositional prover of Task 2 on an equivalent small domain.

Cross-Understanding Requirement

- Student B must explain why Skolemisation preserves satisfiability but not logical equivalence

- Student A must explain what the experimental evidence shows about the cost of lifting resolution to first-order logic, and what the non-entailed query shows about the decidability of first-order entailment

Evidence Expected: Conversion steps, proof tree, prover code, strategy comparison, behaviour on the non-entailed query and responses to the cross-understanding questions.

TASK 6 – Advanced Inference Mini-Investigations (7 Marks)

Both students investigate a different inference strategy on the same knowledge base (an organisation hierarchy with reports-to relations and approval rules, about 10 definite-clause rules, with 100, 1,000 and 10,000 facts).

Student A – Forward Chaining Investigation

- Implement forward chaining to a fixed point
- Record facts inferred, rule firings, redundant derivations, iterations to fixed point and the percentage of derived facts irrelevant to the query
- Explain what happens to the work when the query concerns only a very small part of the KB

Student B – Backward Chaining Investigation

- Implement backward chaining with substitutions
- Engineer a left-recursive rule on which naive backward chaining never terminates, implement a remedy (loop check, depth bound or memoisation) and evaluate its cost
- Record goals expanded, unifications, maximum depth and repeated subgoals

Verification Questions

- Student B must explain: Why is the choice of inference direction central to performance on this KB?
- Student A must explain: Why can backward chaining remain efficient even when the KB contains many facts irrelevant to the query?

Evidence Expected: Two working engines, metric tables for the three fact sizes, the non-termination demonstration, the remedy, and correct responses to the verification questions.

DAY-4 DESIGN CHALLENGE

TASK 7 – Design a Knowledge-Based Reasoning Component from First Principles (10 Marks)

Problem Context – Hospital Medication-Safety Advisor

Given a patient record (age, allergies, current medicines, conditions) and a newly prescribed drug, the agent must decide whether the prescription is safe and explain why. The initial knowledge base has about 40 rules. The hospital plans to grow it to 2,000 rules and 50,000 patients.

This problem must be solved collaboratively. Neither student may work independently and merge the answers later.

Stage 1 – Independent Design Proposal – 1 Mark

Before discussing with each other, Student A and Student B must each independently propose: the representation (propositional or FOL), vocabulary, inference method (forward chaining, backward chaining or resolution) and how explanations will be produced. Each initial proposal must be retained in the submission. No marks are deducted merely because an initial design is incorrect.

Stage 2 – Compare the Two Designs – 1 Mark

Identify where the designs agree and differ, their strengths and weaknesses, and which ideas will be retained.

Stage 3 – Joint Final Design – 3 Marks

Produce the final design: representation, vocabulary, inference method, indexing or agenda strategy for scale, explanation mechanism and complete pseudocode.

Stage 4 – Divide the Validation Responsibilities – 3 Marks

- **Student A – Theoretical Validation:** soundness and completeness of the chosen inference, complexity of a query as a function of KB size, and the effect of restricting the KB to definite clauses.
- **Student B – Experimental Validation:** at least 30 test cases including dangerous combinations, KB sizes of 40, 400 and 2,000 rules, metrics (query latency, inference steps, memory, explanation length) and a baseline comparison, with code and outputs as proof.

Stage 5 – Cross-Defence – 1 Mark

Student A explains the experimental evidence supporting the group's design. Student B explains why the theoretical complexity result applies to the group's chosen inference.

Stage 6 – Joint Evaluation – 1 Mark

Why is the problem suitable for a logical agent? What are the limits of the representation (uncertainty, incomplete data)? What should an AI designer check before choosing a knowledge-based approach?

CONTRIBUTION RECORD

Each pair must include this table at the end of the submission, reflecting the actual work completed.

Task	Student A – Primary Contribution	Student B – Primary Contribution	Joint Work
Task 1			
Task 2			
Task 3			
Task 4			
Task 5			
Task 6			
Task 7			

MARK DISTRIBUTION – ASSIGNMENT 3

Task	Assessed Area	Marks
Task 1	Logical agents, entailment and reasoning patterns	6
Task 2	Model checking versus resolution	7
Task 3	Knowledge engineering in FOL	7
Task 4	Unification and lifted inference	7
Task 5	First-order resolution	6
Task 6	Forward and backward chaining	7
Task 7	Day-4 Design Challenge	10
	TOTAL	**50**

Submission Requirements: one report per pair for Tasks 1–7 with the contribution record, source code for all implementations, raw test data, and the tables/graphs used. Operation counts (models, clauses, inference steps) are primary evidence; time is secondary.

INDIVIDUAL VERIFICATION AND VIVA MODEL

Primary contribution does not imply exclusive responsibility. A student must not answer "My partner did this section." For each student conduct a brief 4–5 minute verification:

1. **Primary responsibility** – one technical question from the component the student led.
2. **Partner's responsibility** – one question from a component primarily completed by the partner.
3. **Transfer question** – modify a condition and ask for the consequence, for example: "If you remove the occurs check, which of your test pairs now gives a wrong answer?" or "If the KB grows to 100,000 facts and queries are rare, would you change from forward to backward chaining?"

Overall Assessment Expectation: Represent → Infer → Implement → Measure → Debug → Evaluate → Defend.

TASK 1 – Logical Agents: Knowledge Bases, Entailment and Reasoning Patterns

Part A – Knowledge Base and Model Checking in the Wumpus World

1. Introduction

The Wumpus World is a simple artificial intelligence environment in which an intelligent agent moves through a grid to find gold while avoiding dangerous objects such as pits and the Wumpus.

The agent uses logical reasoning to identify safe and dangerous squares. It receives information through percepts such as breeze and stench and stores this information in a Knowledge Base (KB).

2. Propositional Symbols

The following propositional symbols are used to represent the Wumpus World:

- $\mathrm{P(i, j)}$: Represents a pit in square (i, j).
- $\mathrm{W(i, j)}$: Represents a Wumpus in square (i, j).
- $\mathrm{B(i, j)}$: Represents a breeze in square $\mathrm{(i, j)}$.
- $\mathrm{S(i, j)}$: Represents a stench in square $\mathrm{(i, j)}$.

These symbols help the agent represent information about every square in the environment.

3. Rules of the Wumpus World

1. A square contains a breeze if at least one of its neighbouring squares contains a pit.
2. A square contains a stench if at least one of its neighbouring squares contains a Wumpus.
3. If a square has no breeze, all its neighbouring squares are free from pits.
4. If a square has no stench, all its neighbouring squares are free from the Wumpus.
5. A square is safe if it contains neither a pit nor a Wumpus.

For example, if square (1,1) has no breeze and no stench, its neighbouring squares can be considered safe from pits and the Wumpus.

4. Number of Propositional Symbols

In an n × n Wumpus World, every square contains four propositional symbols.

Therefore,

$$[\, m = 4n^2]$$

The total number of possible models is:

$$[\, \backslash text\{Models\} = 2^m = 2^{\{4n^2\}}]$$

Grid size	Propositional symbols	Possible models
2 × 2	16	$(2^{\{16\}})$
3 × 3	36	$(2^{\{36\}})$
4 × 4	64	$(2^{\{64\}})$

As the grid size increases, the number of possible models increases very quickly. Therefore, checking every possible model becomes difficult for large environments.

The number of sentences in the Knowledge Base depends on the number of squares, adjacency rules and additional constraints. With a fixed set of rules per square and per neighbouring pair, it grows proportionally to (n^{2}).

5. Relevant Symbols for a Query

When the agent wants to check whether square (2,2) is safe, it does not always need to examine every symbol in the entire environment.

It can focus on the pit and Wumpus symbols of that square and the percept information from its neighbouring squares.

This reduces the number of symbols that must be considered and makes reasoning faster.

6. Soundness and Completeness

Soundness: A reasoning method is sound if every conclusion it produces is logically correct. It never produces a conclusion that does not follow from the Knowledge Base.

Completeness: A reasoning method is complete if it can derive every conclusion that logically follows from the Knowledge Base.

Both properties are important because an intelligent agent must make correct and reliable decisions.

7. Model Checking

Model checking is a method used to determine whether a particular statement is true in every model of a Knowledge Base.

For example, the agent checks whether square (2,2) is safe by examining all possible models that satisfy the available information.

If every model shows that the square is safe, the Knowledge Base entails that the square is safe.

Part B – Reasoning Patterns

1. Introduction

Reasoning patterns are logical methods used by an intelligent agent to derive new information from existing facts and rules.

The main reasoning methods used in this task are Modus Ponens, And-Elimination and Unit Resolution.

2. Modus Ponens

Modus Ponens is a logical inference rule used to derive a conclusion from a condition and its result.

General form:

$$[\, P \rightarrow Q \,]$$

$$[\, P \,]$$

$$[\, \therefore Q \,]$$

Example:

- If there is a breeze, a neighbouring square may contain a pit.
- There is a breeze.
- Therefore, a neighbouring square may contain a pit.

3. And-Elimination

And-Elimination is a rule that allows us to take individual statements from a combined statement.

General form:

$$[\, P \wedge Q \,]$$

$$[\, \therefore P \,]$$

$$[\, \therefore Q \,]$$

Example:

If a square is free from pits and Wumpus, we can separately conclude that it is free from pits and free from the Wumpus.

4. Unit Resolution

Unit Resolution is a logical inference rule that combines clauses to remove opposite literals.

General form:

$$[\ P \vee Q\ ]$$

$$[\ \neg P\ ]$$

$$[\ \therefore Q\ ]$$

Example:

- A square contains a pit or it is free from pits.
- The square does not contain a pit.
- Therefore, it is free from pits.

5. Soundness of Inference Rules

An inference rule is sound if its conclusion is always true whenever its premises are true.

Modus Ponens, And-Elimination and Unit Resolution are sound because they preserve logical truth.

A truth table can be used to verify that whenever the premises are true, the conclusion is also true.

6. Why Not Derived Does Not Mean False

If a statement cannot be derived from the Knowledge Base, it does not automatically mean that the statement is false.

It may simply mean that the Knowledge Base does not contain enough information to prove it.

For example, if the agent does not know whether a square contains a pit, it should consider that square unknown rather than immediately declaring it safe.

7. Model Checking Versus Inference Rules

Model checking examines possible models to determine whether a conclusion follows from the Knowledge Base.

Inference rules derive conclusions by applying logical rules to existing facts.

Inference rules can be more efficient when only a small number of conclusions are required. Model checking provides a systematic method for checking entailment but can become expensive when the number of symbols increases.

TASK 2 – The Quality of Reasoning Method: Model Checking versus Resolution

1. Introduction

Artificial intelligence uses different reasoning methods to solve logical problems. Two important methods are Model Checking and Resolution.

Both methods are used to determine whether a query follows from a given Knowledge Base. However, they use different approaches.

2. Model Checking

Model Checking is a logical reasoning method that checks all possible interpretations of propositional symbols.

It verifies whether a query is true in every model that satisfies the Knowledge Base.

For example, in a smart-building access-control system, the agent checks whether a person is allowed to enter a building based on the available security rules.

Advantages

1. It provides a systematic way to check logical conclusions.
2. It is easy to understand and implement for small Knowledge Bases.
3. It can identify whether a statement is true, false or unknown.
4. It checks all possible models and gives a reliable entailment result.

Disadvantages

1. It becomes slow when the number of propositional symbols increases.
2. It requires checking many possible models.
3. It consumes more computational resources for large problems.

3. Resolution

Resolution is a logical inference technique used to prove whether a statement follows from a Knowledge Base.

It works by converting logical sentences into Conjunctive Normal Form (CNF) and applying resolution rules.

The method usually uses proof by contradiction.

Steps in Resolution

1. Convert the given logical statements into CNF.
2. Negate the query that needs to be proved.

3. Add the negated query to the Knowledge Base.
4. Apply the resolution rule to the clauses.
5. Continue resolving until an empty clause is obtained or no new clauses can be generated.

An empty clause represents a contradiction and proves that the original query is entailed.

Advantages

1. It can prove conclusions without checking every possible model.
2. It is useful for automated theorem proving.
3. It can handle complex logical relationships.
4. It provides a systematic proof procedure.

Disadvantages

1. CNF conversion can produce many clauses.
2. The number of resolution operations may become very large.
3. It requires careful clause management.
4. Some queries may generate a large number of unnecessary clauses.

4. Smart-Building Access-Control Policy

A smart-building system uses logical rules to decide whether a person can enter a building.

The following rules describe the access-control system:

1. A person must have a valid security badge to enter.
2. The person's badge must not be expired.
3. Employees are allowed to enter during working hours.
4. Visitors must have prior approval.
5. Visitors must be accompanied by an authorised employee.
6. A person must not be on the security blacklist.
7. Emergency personnel are allowed during emergency situations.
8. An emergency override can permit entry when normal access conditions are not satisfied.
9. A valid badge and working hours together permit normal employee access.
10. If a person is unauthorised and no emergency override exists, entry is denied.

These rules use logical conditions such as AND, OR, implication and biconditional relationships.

5. Conjunctive Normal Form (CNF)

CNF is a standard form used in propositional logic. It represents a logical expression as a combination of clauses joined by AND, where each clause contains literals joined by OR.

Example:

$$[\ P \rightarrow Q\ ]$$

Removing implication:

$$[\ \neg P \vee Q\ ]$$

Another example:

$$[\ P \rightarrow (Q\backslash land\ R)]$$

CNF conversion:

$$[\ (\neg P \vee Q) \wedge (\backslash neg\ P \vee R)]$$

CNF makes logical expressions suitable for applying resolution.

6. Resolution Proof Example

Consider the following statements:

$$[\ P \rightarrow Q\ ]$$

$$[\ Q \rightarrow R\ ]$$

$$[\ P\ ]$$

We want to prove R.

Convert the implications into CNF:

$$[\ \neg P \vee Q\ ]$$

$$[\ \neg Q \vee R\ ]$$

Add the fact:

$$[\ P\ ]$$

By resolution between P and the first clause:

$$[\ Q\ ]$$

By resolution between Q and the second clause:

$$[\ R\ ]$$

Therefore, R is proved.

7. Model Checking and Resolution Comparison

Model Checking	Resolution
Checks possible models.	Applies logical inference rules.
Uses truth assignments.	Uses CNF clauses.
Suitable for small Knowledge Bases.	Useful for automated theorem proving.
Can become expensive due to many models.	Can become expensive due to many clauses.
Checks whether a query is entailed.	Proves entailment using contradiction.

8. When Is Model Checking Faster?

Model checking can be faster for small Knowledge Bases with a limited number of propositional symbols.

For example, if a problem contains only a few variables, checking all possible models may be easier than converting expressions into CNF and generating resolution clauses.

9. When Is Resolution Faster?

Resolution can be faster when the Knowledge Base contains a large number of possible assignments but the required conclusion can be proved using only a small number of clauses.

For example, a long implication chain can sometimes be resolved directly without examining all possible truth assignments.

10. Factors to Consider Before Selecting a Reasoning Method

Before selecting a reasoning method, an AI designer should consider:

1. The number of logical symbols and rules.
2. The complexity of the Knowledge Base.
3. The type of queries that need to be answered.
4. Available memory and processing power.
5. The expected number of inference operations.
6. The accuracy and reliability requirements.
7. The possibility of generating unnecessary clauses or models.

TASK 3 – Breaking the Expressiveness Barrier: Knowledge Engineering in First-Order Logic

1. Introduction

First-Order Logic (FOL) is a knowledge representation technique used in artificial intelligence to represent objects, properties and relationships.

It is more expressive than propositional logic because it uses variables, predicates and quantifiers.

In this task, First-Order Logic is used to develop a University Exam-Eligibility Advisor that determines whether a student is eligible to write an examination.

2. Problem Description

A university has several rules for deciding whether students are eligible to write examinations.

The eligibility decision depends on:

1. Minimum attendance percentage.
2. Completion of required prerequisites.
3. Payment of examination fees.
4. Disciplinary status of the student.
5. Approval of leave, if applicable.

The system checks these conditions and provides an eligibility decision.

3. Knowledge Engineering Process

Knowledge Engineering is the process of collecting, organising and representing knowledge so that an intelligent system can use it for reasoning.

The following steps are involved:

Step 1 – Identify the Task: Define the purpose of the system. Here, the purpose is to determine examination eligibility.

Step 2 – Assemble Knowledge: Collect university examination regulations, attendance requirements, fee rules and disciplinary rules.

Step 3 – Decide the Vocabulary: Identify the objects, predicates and relationships required to represent the regulations.

Step 4 – Encode General Knowledge: Convert university rules into logical statements.

Step 5 – Encode the Instance: Add individual student details such as attendance, fees and completed subjects.

Step 6 – Pose Queries: Ask questions such as whether a particular student is eligible to write an examination.

Step 7 – Debug the Knowledge Base: Identify incorrect or missing rules and correct them to obtain proper results.

4. Propositional Logic Representation

Propositional logic represents each condition using a separate statement.

For example:

- A: Student has sufficient attendance.
- P: Student has completed prerequisites.
- F: Student has paid fees.
- D: Student has no disciplinary restriction.
- L: Student has approved leave.
- E: Student is eligible.

The eligibility rule can be represented as:

$$[\, E = A \wedge P \wedge F \wedge D \,]$$

If approved leave is allowed as an alternative to the attendance requirement, the rule becomes:

$$[\, E = (A \vee L) \wedge P \wedge F \wedge D \,]$$

This representation is simple but requires separate rules for different students and courses.

5. First-Order Logic Vocabulary

The following predicates are used:

Predicate	**Meaning**
Student(x)	x is a student
Course(c)	c is a course
Attendance(x, c, a)	Attendance of x in c is a
Prerequisite Cleared(x, c)	Student cleared prerequisites
Fees Paid(x, c)	Student paid examination fees
Disciplinary Clear(x)	Student has no disciplinary restriction

Approved Leave(x, c)	Student has approved leave
Eligible(x, c)	Student is eligible
Required Attendance(c, a)	Course attendance requirement
Requires(c, p)	Course c requires prerequisite p
Passed(x, p)	Student passed prerequisite p
Enrolled(x, c)	Student is enrolled in course c

6. First-Order Logic Axioms

The following axioms represent the university examination rules.

Axiom 1: Every student must be a registered student.

$$[\ \forall x\big(\text{Student}(x) \rightarrow \text{Registered}(x)\big)]$$

Axiom 2: Every course must be a valid course.

$$[\ \forall c\big(\text{Course}(c) \rightarrow \text{ValidCourse}(c)\big)]$$

Axiom 3: A student with sufficient attendance satisfies the attendance requirement.

$$[\ \forall x \forall c\big(\text{SufficientAttendance}(x, c) \rightarrow \text{AttendanceOK}(x, c)\big)]$$

Axiom 4: A student with approved leave satisfies the attendance exemption condition.

$$[\ \forall x \forall c\big(\text{ApprovedLeave}(x, c) \rightarrow \text{AttendanceOK}(x, c)\big)]$$

Axiom 5: A student must clear all required prerequisites.

$$[\ \forall x \forall c\big(\text{PrerequisiteCleared}(x, c) \rightarrow \text{PrerequisiteOK}(x, c)\big)]$$

Axiom 6: A student who has paid the examination fees satisfies the fee condition.

$$[\ \forall x \forall c\big(\text{FeesPaid}(x, c) \rightarrow \text{FeeOK}(x, c)\big)]$$

Axiom 7: A student without disciplinary restrictions satisfies the disciplinary condition.

$$[\ \forall x\big(\text{DisciplinaryClear}(x) \rightarrow \text{DisciplineOK}(x)\big)]$$

Axiom 8: A student who satisfies all examination conditions is eligible.

$$[\ \forall x \forall c[\ \text{AttendanceOK}(x, c) \wedge \text{PrerequisiteOK}(x, c)\ ]$$

$$[\ \wedge\ \text{FeeOK}(x, c) \wedge \text{DisciplineOK}(x)\ \rightarrow \text{Eligible}(x, c)]\ ]$$

Axiom 9: Every eligible student must be enrolled in the course.

$$[\ \forall x \forall c\big(\text{Eligible}(x, c) \rightarrow \text{Enrolled}(x, c)\big)]$$

Axiom 10: Every student enrolled in a course has a student record.

$$[\ \forall x \forall c\big(\text{Enrolled}(x, c) \rightarrow \text{HasRecord}(x, c)\big)]$$

Axiom 11: Every course must have at least one examination.

$$[\ \forall c\ \Big(\text{Course}(c) \rightarrow \backslash\text{exists}\ e\big(\text{Exam}(e, c)\big)\Big)]$$

Axiom 12: Every student has at least one academic record.

$$[\ \forall x\ \Big(\text{Student}(x) \rightarrow \backslash\text{exists}\ r\big(\text{AcademicRecord}(r, x)\big)\Big)]$$

These axioms use universal and existential quantifiers to represent general university knowledge.

7. Existential and Universal Quantifiers

Universal Quantifier (∀): It means that a rule applies to every object.

Example:

$$[\ \forall x\big(\text{Student}(x) \rightarrow \text{Registered}(x)\big)]$$

It means every student must be registered.

Existential Quantifier (∃): It means that at least one object exists.

Example:

$$[\ \exists x\big(\text{Student}(x) \wedge \text{Eligible}(x, \text{AI})\big)]$$

It means at least one student is eligible for the AI examination.

8. Propositional Logic Versus First-Order Logic

Propositional Logic	**First-Order Logic**
Uses simple statements.	Uses objects, predicates and variables.
Does not use quantifiers.	Uses universal and existential quantifiers.
Requires separate rules for different cases.	Uses general rules for multiple students.
Less expressive.	More expressive.
Suitable for small problems.	Suitable for complex knowledge representation.

9. Growth of Knowledge Base

In propositional logic, separate rules are required for every student and course.

If there are s students and c courses, the number of student-course combinations is:

$$[\, s \times c \,]$$

Therefore, the number of rules grows approximately as:

$$[\, O(sc) \,]$$

In First-Order Logic, general rules can be reused for different students and courses.

The number of general axioms can remain constant while individual facts increase.

For example, the same eligibility rule can be applied to 10 students or 1,000 students.

10. Competency Queries

The following queries can be used to test the eligibility system.

Five successful queries:

1. Is student S1 eligible for AI?
2. Has student S2 paid the examination fees?
3. Has student S3 cleared all prerequisites?
4. Does student S4 have sufficient attendance?
5. Is student S5 free from disciplinary restrictions?

Five unsuccessful queries:

1. Is student S6 eligible without sufficient attendance or approved leave?
2. Is student S7 eligible without clearing prerequisites?
3. Is student S8 eligible without paying fees?
4. Is student S9 eligible with a disciplinary restriction?
5. Is an unregistered student eligible for an examination?

Five queries returning bindings:

1. Find all students eligible for AI.
2. Find all students who have paid their fees.
3. Find all students who have cleared prerequisites.
4. Find all students with approved leave.
5. Find all courses in which a particular student is eligible.

11. Possible Knowledge Base Defects and Fixes

1. **Missing attendance rule:** The system may not check attendance correctly. Fix it by adding the minimum attendance condition.
2. **Incorrect fee information:** A student may be shown as eligible even though fees are unpaid. Fix it by updating the fee records.
3. **Missing prerequisite information:** The system may approve students who have not completed required subjects. Fix it by adding prerequisite relationships.

12. Why Greater Expressiveness Does Not Always Improve Performance

First-Order Logic can represent more complex information than propositional logic. However, it may require additional processing for variable matching, unification and quantifier handling.

For small Knowledge Bases, propositional logic may be faster because it uses simple truth values.

For large Knowledge Bases, First-Order Logic can be more efficient because general rules can be reused.

TASK 4 – The Reality of Unification and Lifted Inference

1. Introduction

Unification is an important process in First-Order Logic. It is used to identify whether two logical expressions can be made identical by replacing variables with suitable terms.

Lifted inference uses variables and substitutions to perform reasoning without generating every possible ground instance.

These methods help artificial intelligence systems perform logical reasoning efficiently.

2. Meaning of Unification

Unification is the process of finding substitutions that make two expressions identical.

For example:

$$[\ \text{Knows}(x, \text{John})]$$

$$[\ \text{Knows}(\text{Mary}, y)]$$

The expressions can be made identical by using:

$$[\, x = \text{Mary} \,]$$

$$[\, y = \text{John} \,]$$

Therefore, the expressions are successfully unified.

3. Most General Unifier (MGU)

The Most General Unifier is the simplest substitution that makes two expressions identical without adding unnecessary restrictions.

Example:

$$[\, \text{Likes}(x, \text{Apple})]$$

$$[\, \text{Likes}(\text{Ravi}, y)]$$

The substitutions are:

$$[\, x = \text{Ravi}, \backslash\text{quad}\, y = \text{Apple} \,]$$

Therefore, the MGU is:

$$[\frac{\{x}{\text{Ravi}}, \frac{y}{\text{Apple}\}}]$$

4. Five Examples of Unification

Example 1:

$$[\, \text{Knows}(x, \text{John})]$$

$$[\, \text{Knows}(\text{Ravi}, y)]$$

MGU:

$$[\frac{\{x}{\text{Ravi}}, \frac{y}{\text{John}\}}]$$

Example 2:

$$[\, \text{Likes}(x, x)]$$

$$[\, \text{Likes}(\text{Apple}, \text{Apple})]$$

MGU:

$$[\frac{\{x}{\text{Apple}\}}]$$

Example 3:

$$[\, \text{Parent}(x, \text{Mother})]$$

$$[\ \text{Parent}(\text{Ravi}, y)]$$

MGU:

$$[\frac{\{x}{\text{Ravi}}, \frac{y}{\text{Mother}\}}]$$

Example 4 – Occurs Check:

$$[\ P(x)]$$

$$[\ P\big(f(x)\big)]$$

These expressions cannot be unified because x would need to contain itself.

The occurs check prevents this invalid substitution.

Example 5 – Standardising Apart:

Consider:

$$[\ \text{Parent}(x, y)]$$

$$[\ \text{Parent}(\text{John}, x)]$$

The variable x in the second expression must first be renamed to a new variable z.

The expressions become:

$$[\ \text{Parent}(x, y)]$$

$$[\ \text{Parent}(\text{John}, z)]$$

MGU:

$$[\frac{\{x}{\text{John}}, \frac{y}{z\}}]$$

Standardising apart prevents variables from different logical rules from being treated as the same variable accidentally.

5. Substitution Composition

Substitution composition means combining multiple substitutions into a single substitution.

For example:

First substitution:

$$[\ \theta_1 = \frac{\{x}{y\}}]$$

Second substitution:

$$[\theta_2 = \frac{\{y}{John\}}]$$

After applying both substitutions:

$$[\theta = \frac{\{x}{John}, \frac{y}{John\}}]$$

This process helps maintain consistency during logical inference.

6. Occurs Check

The occurs check is used to verify that a variable does not appear inside the term being assigned

For example:

$$[x = f(x)]$$

This substitution is rejected because x appears inside f(x).

The occurs check prevents infinite or invalid substitutions.

7. Lifted Modus Ponens

Lifted Modus Ponens is an inference rule used in First-Order Logic.

It applies Modus Ponens to expressions containing variables by using unification.

General form:

$$[P(x) \rightarrow Q(x)]$$

$$[P(John)]$$

Therefore:

$$[Q(John)]$$

Here, the variable x is replaced with John through unification.

This method allows general rules to be applied to different objects without separately writing every possible instance.

8. Propositionalisation

Propositionalisation is the process of converting First-Order Logic statements into propositional statements by replacing variables with specific ground terms.

Example:

$$[\forall x\big(Student(x) \rightarrow Learns(x)\big)]$$

If the constants are Ravi and Sita, the rule becomes:

$$[\text{Student(Ravi)} \rightarrow \text{Learns(Ravi)}]$$

$$[\text{Student(Sita)} \rightarrow \text{Learns(Sita)}]$$

This creates separate ground instances for each constant.

9. Growth of Ground Clauses

The number of possible ground instances increases when the number of constants increases.

If a predicate has arity r and there are k constants, the number of possible ground atomic expressions is:

$$[k^r]$$

Examples:

Constants (k)	Arity 1	Arity 2	Arity 3
3	3	9	27
5	5	25	125
10	10	100	1,000
20	20	400	8,000
30	30	900	27,000

This shows that ground expressions increase rapidly when predicate arity increases.

10. Effect of Function Symbols

Function symbols can generate new terms from existing constants.

For example:

$$[f(a), f(f(a)), f(f(f(a)))]$$

If the maximum term depth increases, the number of possible ground terms also increases.

Function symbols can create an unlimited number of terms if no depth restriction is applied.

Therefore, they may increase the complexity of logical reasoning.

11. Lifted Inference Versus Propositional Inference

Lifted Inference	Propositional Inference
Uses variables and substitutions.	Uses ground statements.

Works directly with general rules.	Generates separate instances.
Avoids unnecessary ground instances.	May generate many ground instances.
Requires unification.	Does not require unification.
Useful for large structured Knowledge Bases.	Useful for small finite domains.

12. Why Lifted Inference Is Sound

Lifted inference is sound because it applies valid logical inference rules using correct substitutions.

If the original premises are true, the derived conclusion must also be true.

Unification ensures that the expressions match correctly before inference is performed.

13. When Should We Use Propositionalisation?

Propositionalisation may be preferred when:

1. The domain contains only a small number of objects.
2. The number of ground instances is manageable.
3. The Knowledge Base contains simple relationships.
4. Fast processing is more important than general representation.
5. The system requires a finite and fixed set of facts.

TASK 5 – Upgrading Resolution from Propositional to First-Order Logic

1. Introduction

First-Order Resolution is a logical reasoning method used to prove statements involving objects, variables and relationships.

It extends propositional resolution by using unification to match logical expressions.

In this task, First-Order Resolution is used to prove that an authorised employee exists based on information about restricted documents and audited reports.

2. Given Statements

The problem provides the following English statements:

1. Every employee who handled a restricted document is authorised.
2. Every document containing patient data is restricted.
3. Some employee handled every document that was audited.
4. Report R1 contains patient data and R1 was audited.

The objective is to prove that an authorised employee exists.

3. First-Order Logic Representation

The following predicates are used:

- Employee(x): x is an employee.
- Handled(x,y): Employee x handled document y.
- Restricted(y): Document y is restricted.
- PatientData(y): Document y contains patient data.
- Audited(y): Document y was audited.
- Authorised(x): Employee x is authorised.

The statements are converted into First-Order Logic.

Statement 1:

Every employee who handled a restricted document is authorised.

$$[\,\forall x \forall y[\,Employee(x) \wedge Handled(x,y) \wedge Restricted(y) \rightarrow Authorised(x)]]$$

***Statement* 2**:

Every document containing patient data is restricted.

$$[\,\forall y[\,\mathrm{PatientData(y)} \rightarrow \mathrm{Restricted(y)}]]$$

Statement 3:

Some employee handled every document that was audited.

$$[\,\exists x \forall y[\,\mathrm{Audited(y)} \rightarrow \mathrm{Handled(x,y)}]]$$

Statement 4:

Report R1 contains patient data and was audited.

$$[\ \text{PatientData}(R1)]$$

$$[\ \text{Audited}(R1)]$$

4. Skolemisation

Skolemisation is the process of removing existential quantifiers by replacing existential variables

The third statement contains an existential quantifier:

$$[\ \exists x \forall y[\ \text{Audited}(y) \rightarrow \text{Handled}(x, y)]]$$

Since x does not depend on any universally quantified variable, it can be replaced with a Skolem c

Therefore:

$$[\ \forall y[\ \text{Audited}(y) \rightarrow \text{Handled}(E1, y)]]$$

The employee E1 represents an employee who handled every audited document.

5. Conversion into CNF

CNF conversion involves removing implications and converting logical statements into clauses.

Clause 1 — Statement 1:

Starting expression:

$$[\ \text{Employee}(x) \wedge \text{Handled}(x, y) \wedge \text{Restricted}(y) \rightarrow \text{Authorised}(x)]$$

After removing implication:

$$[\ \neg\text{Employee}(x) \vee \neg\text{Handled}(x, y)]$$

$$[\ \vee \neg\text{Restricted}(y) \vee \text{Authorised}(x)]$$

Clause 2 — Statement 2:

$$[\ \text{PatientData}(y) \rightarrow \text{Restricted}(y)]$$

CNF:

$$[\ \neg\text{PatientData}(y) \vee \text{Restricted}(y)]$$

Clause 3 — Statement 3:

After Skolemisation:

$$[\ \text{Audited}(y) \rightarrow \text{Handled}(E1, y)]$$

CNF:

$$[\ \neg\text{Audited}(y) \vee \text{Handled}(E1, y)]$$

Clause 4 − Statement 4:

$$[\ \text{PatientData}(R1)]$$

Clause 5 − Statement 4:

$$[\ \text{Audited}(R1)]$$

Clause 6 − Employee Fact:

The existential employee E1 is represented as an employee:

$$[\ \text{Employee}(E1)]$$

6. Resolution Proof

The objective is:

$$[\ \exists x, \text{Authorised}(x)]$$

To prove this using resolution, negate the query and add it to the Knowledge Base.

Negated query:

$$[\ \neg\exists x, \text{Authorised}(x)]$$

Equivalent form:

$$[\ \forall x,\backslash\text{neg Authorised}(x)]$$

Now apply resolution.

Step 1: From Clause 2 and the patient − data fact:

$$[\ \neg\text{PatientData}(R1) \vee \text{Restricted}(R1)]$$

Since PatientData(R1)is true:

$$[\ \text{Restricted}(R1)]$$

Step 2: From Clause 3 and the audited fact:

$$[\ \neg\text{Audited}(R1) \vee \text{Handled}(E1, R1)]$$

Since Audited(R1)is true:

$$[\ \text{Handled}(E1, R1)]$$

Step 3: We already have:

$$[\ \text{Employee}(E1)]$$

$$[\ \text{Handled}(E1, R1)]$$

$$[\ \text{Restricted}(R1)]$$

Using Clause 1, we derive:

$$[\ \text{Authorised}(E1)]$$

Step 4: The negated query contains:

$$[\ \neg\text{Authorised}(E1)]$$

Resolving both statements produces:

$$[\ \Box\ ]$$

The empty clause represents a contradiction.

Therefore:

$$[\ \boxed{\exists x, \text{Authorised}(x)}]$$

Hence, an authorised employee exists.

7. Proof Tree

$$\textbf{PatientData}(\textbf{R1})$$

$$\textbf{Audited}(\textbf{R1})$$

$$\textbf{Restricted}(\textbf{R1})$$

From patient − data rule

$$\textbf{Handled}(\textbf{E1}, \textbf{R1})$$

From audited − document rule

$$\textbf{Employee}(\textbf{E1})$$

$$\textbf{Authorised}(\textbf{E1})$$

Proved

An authorised employee exists.

8. Importance of Unification in First − Order Resolution

Unification is required to match variables with constants or other terms during resolution.

For example:

$$[\ \text{Audited}(y)]$$

$$[\ \text{Audited}(R1)]$$

The unifier is:

$$\left[\frac{\{y}{R1\}}\right]$$

After applying this substitution, both expressions become identical.

9. Skolemisation: Satisfiability and Logical Equivalence

Skolemisation replaces existential variables with Skolem constants or functions.

It preserves satisfiability, meaning the original expression and its Skolemised form are either both satisfiable or both unsatisfiable.

However, they are not logically equivalent in the ordinary sense because the Skolemised expression introduces a new constant or function symbol.

It is mainly used to simplify First-Order Logic expressions for resolution.

10. First-Order Resolution Versus Propositional Resolution

Propositional Resolution	**First-Order Resolution**
Uses propositional symbols.	Uses predicates and variables.
Does not require unification.	Requires unification.
Works with fixed statements.	Works with objects and relationships.
Simpler to implement.	More complex to implement.
Limited expressiveness.	Greater expressiveness.

11. Non-Entailed Queries and Decidability

A query is non-entailed when the available Knowledge Base does not logically prove it.

For example, the given information does not establish that a particular employee named Ravi is authorised.

A First-Order Resolution prover may continue generating clauses without reaching an empty clause.

First-Order Logic entailment is semi-decidable: if a statement is entailed, a complete resolution procedure can eventually find a proof. If it is not entailed, the procedure may continue indefinitely.

Therefore, a step limit can be used in practical systems to control computational cost.

TASK 6 – Advanced Inference Mini-Investigations

1. Introduction

Forward Chaining and Backward Chaining are two important inference methods used in artificial intelligence.

They help intelligent systems derive new facts from existing facts and rules.

Both methods can be used in an organisation hierarchy to identify reporting relationships and approval permissions.

2. Organisation Hierarchy

An organisation contains employees working at different positions.

For example:

- Manager
- Team Leader
- Employee
- Department Head
- General Manager

The system stores relationships between employees and uses logical rules to determine reporting structures and approval permissions.

3. Knowledge Base

A Knowledge Base contains facts and rules about the organisation.

Example facts:

1. Ravi reports to Kumar.
2. Kumar reports to Suresh.
3. Suresh is a department head.
4. Ravi is an employee.
5. Kumar is a team leader.
6. Suresh is a manager.

Example rules:

1. If employee A reports to B, B is the supervisor of A.
2. If A reports to B and B reports to C, C is a higher-level supervisor of A.

3. Every department head is a manager.
4. Every manager can approve department requests.
5. A supervisor can review employee performance.
6. A manager can approve leave requests.
7. A department head can approve department expenses.
8. Every employee belongs to an organisation.
9. An employee's supervisor can review their work.
10. A higher-level supervisor can review requests from lower-level employees, subject to the organisation's approval policy.

Part A – Forward Chaining Investigation

4. Meaning of Forward Chaining

Forward Chaining is a data-driven reasoning method.

It starts with known facts and repeatedly applies logical rules to generate new facts until no more conclusions can be derived.

It is useful when the system has a large amount of input information and needs to discover all possible conclusions.

5. Working Process of Forward Chaining

1. Load all known facts into the Knowledge Base.
2. Check which rules match the available facts.
3. Apply the matching rules.
4. Generate new facts.
5. Add the new facts to the Knowledge Base.
6. Repeat the process until no new facts are generated.

6. Example of Forward Chaining

Given facts:

$$[\ \mathrm{ReportsTo}(\mathrm{Ravi}, \mathrm{Kumar})]$$

$$[\ \mathrm{ReportsTo}(\mathrm{Kumar}, \mathrm{Suresh})]$$

Rule 1:

$$[\ \mathrm{ReportsTo}(x, y) \rightarrow \mathrm{Supervisor}(y, x)]$$

From this rule:

$$[\ \text{Supervisor}(\text{Kumar}, \text{Ravi})]$$

$$[\ \text{Supervisor}(\text{Suresh}, \text{Kumar})]$$

Rule 2:

$$[\ \text{ReportsTo}(x, y) \land \text{ReportsTo}(y, z) \rightarrow \text{HigherSupervisor}(z, x)]$$

Therefore:

$$[\ \text{HigherSupervisor}(\text{Suresh}, \text{Ravi})]$$

The system derives new information from the available facts.

7. Fixed Point

A fixed point is reached when applying the rules does not produce any new facts.

At this stage, the Forward Chaining process stops.

The Knowledge Base contains all the facts that can be derived using the available rules.

8. Performance Metrics

The performance of Forward Chaining can be evaluated using:

- Number of facts inferred.
- Number of rule firings.
- Number of redundant derivations.
- Number of iterations required to reach a fixed point.
- Percentage of derived facts that are irrelevant to the query.

When the Knowledge Base becomes large, Forward Chaining may perform unnecessary work because it derives facts that are not required for a particular query.

9. Limitations of Forward Chaining

1. It may generate many unnecessary facts.
2. It can consume more memory.
3. It may require many rule evaluations.
4. It may become slower when the Knowledge Base is large.
5. It performs unnecessary work when only a small part of the Knowledge Base is relevant.

Part B – Backward Chaining Investigation

10. Meaning of Backward Chaining

Backward Chaining is a goal-driven reasoning method.

It starts with a query or goal and works backwards to determine whether the available facts and rules can prove that goal.

Instead of deriving every possible fact, it searches only for information required to answer the query.

11. Working Process of Backward Chaining

1. Start with a goal or query.
2. Search for a rule that can prove the goal.
3. Identify the conditions required by that rule.
4. Treat each condition as a new subgoal.
5. Search for facts or rules that satisfy the subgoals.
6. Continue until all required conditions are proved or fail.

12. Example of Backward Chaining

Suppose the query is:

$$[\ \mathrm{HigherSupervisor}(\mathrm{Suresh}, \mathrm{Ravi})]$$

The system searches for a rule that can prove this statement.

Rule:

$$[\ \mathrm{ReportsTo}(x, y) \land \mathrm{ReportsTo}(y, z)]$$

$$[\ \rightarrow \mathrm{HigherSupervisor}(z, x)]$$

The system now checks:

$$[\ \mathrm{ReportsTo}(\mathrm{Ravi}, \mathrm{Kumar})]$$

and

$$[\ \mathrm{ReportsTo}(\mathrm{Kumar}, \mathrm{Suresh})]$$

Both facts are available.

Therefore:

$$[\ \mathrm{HigherSupervisor}(\mathrm{Suresh}, \mathrm{Ravi})]$$

The goal is successfully proved.

13. Left Recursion Problem

Left recursion occurs when a rule directly or indirectly refers to itself before reaching a final fact.

Example:

$$[\,A(x) \rightarrow A(x)]$$

If backward chaining repeatedly applies this rule, it may continue forever without reaching a solution.

This is called a non-termination problem.

14. Methods to Prevent Non-Termination

Loop Checking: The system checks whether the same goal has already appeared in the current reasoning path. If it appears again, the system stops that repeated search.

Depth Bound: A maximum reasoning depth is specified. The system stops searching when the limit is reached.

Memoisation: Previously solved goals and their results are stored so that repeated calculations can be avoided.

These methods improve the performance and reliability of Backward Chaining.

15. Forward Chaining Versus Backward Chaining

Forward Chaining	**Backward Chaining**
Data-driven method.	Goal-driven method.
Starts with known facts.	Starts with a query.
Generates new conclusions.	Searches for supporting facts.
May derive unnecessary information.	Focuses on relevant information.
Useful for monitoring and rule-based systems.	Useful for question answering and diagnosis.
May consume more memory.	Can reduce unnecessary processing.

16. Why Is the Choice of Inference Direction Important?

The choice of inference direction is important because it directly affects the number of rules and facts that need to be processed.

Forward Chaining is suitable when the system needs to generate many possible conclusions from available facts.

Backward Chaining is suitable when the system needs to answer a specific question.

Selecting the correct method reduces unnecessary calculations and improves system performance.

17. Why Can Backward Chaining Be Efficient with Large Knowledge Bases?

Backward Chaining can remain efficient even when the Knowledge Base contains many irrelevant facts because it only searches for facts and rules related to the given query.

For example, if the system needs to find Ravi's supervisor, it does not need to examine every employee's salary, attendance or department information.

It focuses only on the reporting relationships required to prove the query.

Conclusion

Forward Chaining and Backward Chaining are important reasoning techniques in artificial intelligence. Forward Chaining starts with facts and generates conclusions, while Backward Chaining starts with a goal and searches for supporting information. The selection depends on the problem, Knowledge Base size and type of query.

TASK 7 – Design a Knowledge-Based Reasoning Component from First Principles

Problem Context – Hospital Medication-Safety Advisor

1. Introduction

A Hospital Medication-Safety Advisor is an intelligent system designed to check whether a prescribed medicine is safe for a patient.

The system uses patient information such as age, allergies, current medicines and medical conditions to make a decision.

It analyses the prescription using logical rules and provides an explanation for its decision.

The main objective is to reduce medication errors and improve patient safety.

Stage 1 – Independent Design Proposal

Student A – Initial Design Proposal

Representation: First-Order Logic.

Vocabulary:

- Patient(x): Represents a patient.
- Drug(d): Represents a medicine.
- Allergy(x,d): Patient has an allergy to a drug.
- Takes(x,d): Patient currently takes a drug.
- Safe(x,d): Drug is safe for the patient.
- Unsafe(x,d): Drug is unsafe for the patient.
- Age(x,a): Represents the patient's age.

Inference Method: Backward Chaining.

Explanation Method: The system provides reasons for its decision by identifying the rules and patient information used.

Student B – Initial Design Proposal

Representation: First-Order Logic.

Vocabulary:

- Patient(x)
- Prescribed(x,d)
- Allergy(x,d)
- Interaction(d1,d2)
- Condition(x,c)
- Contraindicated(d,c)
- Safe(x,d)
- Unsafe(x,d)

Inference Method: Forward Chaining.

Explanation Method: The system stores the rules applied and the facts used to reach the final decision.

Stage 2 – Compare the Two Designs

Student A	**Student B**
Uses Backward Chaining.	Uses Forward Chaining.

Starts with a safety query.	Starts with patient facts.
Checks relevant conditions.	Derives possible safety conclusions.
Avoids unnecessary rule checking.	Can identify multiple safety problems.
Provides goal-based explanations.	Provides fact-based explanations.

Final Decision

The final system uses First-Order Logic with Backward Chaining because the main purpose is to answer a specific question: whether a prescribed drug is safe for a particular patient.

Forward Chaining can also be used for generating safety alerts when new patient information is entered.

Stage 3 – Joint Final Design

1. Representation

First-Order Logic is selected because it can represent patients, medicines, allergies, medical conditions and relationships.

General rules can be reused for multiple patients and medicines.

2. Vocabulary

The following predicates are used:

Predicate	**Meaning**
Patient(x)	x is a patient
Drug(d)	d is a medicine
Prescribed(x,d)	Drug d is prescribed to patient x
Allergy(x,d)	Patient x has an allergy to drug d
Takes(x,d)	Patient x currently takes drug d
Interaction(d1,d2)	Two medicines interact
Condition(x,c)	Patient has a medical condition
Contraindicated(d,c)	Drug is unsuitable for a condition
Age(x,a)	Patient's age

Safe(x,d)	Drug is considered safe under checked rules
Unsafe(x,d)	Drug is considered unsafe
Review Required(x,d)	Prescription requires professional review

3. Knowledge Base Rules

The following rules are used in the medication-safety system.

Rule 1: A drug that causes an allergy in a patient is unsafe for that patient.

$$[\,\text{Allergy}(x, d) \rightarrow \text{Unsafe}(x, d)]$$

Rule 2: A drug that interacts dangerously with another medicine taken by the patient is unsafe.

$$[\,\text{Takes}(x, d_2) \wedge \text{Interaction}(d, d_2)]$$

$$[\rightarrow \text{Unsafe}(x, d)]$$

Rule 3: A drug contraindicated for a patient's medical condition is unsafe.

$$[\,\text{Condition}(x, c) \wedge \text{Contraindicated}(d, c)]$$

$$[\rightarrow \text{Unsafe}(x, d)]$$

Rule 4: A prescription involving an unknown allergy status requires professional review.

Rule 5: A medicine with an age restriction requires age verification.

Rule 6: If a medicine interacts with another current medicine, the system generates an interaction alert.

Rule 7: If patient information is incomplete, the prescription requires review.

Rule 8: If a prescription contains a dangerous drug combination, the system generates a warning.

Rule 9: If a medicine is unsuitable for the patient's condition, the system recommends professional review.

Rule 10: If no known safety issue is identified but important information is missing, the system does not automatically declare the medicine safe.

Rule 11: If a medicine passes all required checks using verified information, it can be classified as having no identified contraindication under the available rules.

Rule 12: Every unsafe prescription must have at least one recorded reason.

Rule 13: Every prescription must be associated with a valid patient record.

Rule 14: Every prescribed medicine must be a recognised drug.

Rule 15: If a prescription requires review, the system must refer it to a qualified healthcare professional.

4. Inference Method

Backward Chaining is selected as the main inference method.

The system starts with the goal of checking whether a prescribed medicine is safe.

It searches for relevant rules and verifies the required patient information.

For example, if the system wants to check whether a medicine is safe, it examines allergies, existing medicines, medical conditions and age-related restrictions.

5. Indexing and Agenda Strategy

Indexing is used to organise the Knowledge Base so that relevant rules can be found quickly.

The system maintains separate indexes for:

- Patient records.
- Medicine information.
- Allergy records.
- Drug interactions.
- Medical conditions.
- Prescription records.

An agenda strategy maintains the list of pending goals and safety checks.

This reduces unnecessary searching and improves performance when the system contains thousands of rules.

6. Explanation Mechanism

The system must explain why a medicine is considered safe, unsafe or requires review.

For example:

Decision: Unsafe.

Reason: The patient has a recorded allergy to the prescribed medicine.

Action: Do not proceed without review by a qualified healthcare professional.

The system stores the rules and facts responsible for each decision.

7. Complete Pseudocode

The working process of the Hospital Medication-Safety Advisor is explained below.

1. Start the system.
2. Load the Knowledge Base.
3. Receive patient information.
4. Receive the prescribed medicine.
5. Check whether the patient record is valid.
6. Check the medicine information.
7. Verify the patient's allergy records.
8. Check current medicines for dangerous interactions.
9. Check the patient's medical conditions.
10. Verify age-related restrictions.
11. Apply the logical rules using Backward Chaining.
12. If a dangerous condition is found, classify the prescription as unsafe.
13. If important information is missing, classify the prescription as requiring review.
14. If all required checks are completed without an identified contraindication, report that no known safety issue was found under the available rules.
15. Display the decision and its explanation.
16. Stop the system.

Stage 4 – Divide the Validation Responsibilities

Student A – Theoretical Validation

1. Soundness

Soundness means that every conclusion produced by the reasoning system must logically follow from the Knowledge Base.

For example, if a patient has a confirmed allergy to a particular medicine and the rule states that such a medicine is unsafe, the system can correctly conclude that the prescription is unsafe.

2. Completeness

Completeness means that the reasoning system can derive every conclusion that logically follows from the Knowledge Base.

A complete reasoning method should be able to identify all safety problems that are represented by the available rules.

3. Complexity of Query

The complexity of a query depends on the number of rules, facts and relationships that must be examined.

When the number of rules increases, the system may require more processing time.

Indexing and Backward Chaining can reduce unnecessary searching.

4. Effect of Definite Clauses

Definite clauses contain exactly one positive literal.

They are useful for rule-based reasoning and can be processed efficiently using Forward Chaining and Backward Chaining.

However, restricting the Knowledge Base to definite clauses limits the types of logical relationships that can be represented directly.

For example, some complex negative and disjunctive rules require additional representation methods.

Student B – Experimental Validation

The system should be tested using at least 30 different test cases.

The test cases should include:

1. Patient with no known allergies.
2. Patient with a known drug allergy.
3. Patient taking multiple medicines.
4. Patient with a dangerous drug interaction.
5. Patient with a medical condition that contraindicates a medicine.
6. Patient with missing age information.
7. Patient with incomplete medical records.
8. Patient receiving a medicine with age restrictions.
9. Patient with multiple allergies.
10. Patient taking a medicine with no known interaction.
11. Patient with an unknown allergy status.
12. Patient with a missing prescription record.
13. Patient with incorrect medicine information.
14. Patient with multiple medical conditions.

15. Patient with an incomplete medicine history.
16. Patient with a confirmed contraindication.
17. Patient with a possible drug interaction.
18. Patient receiving a medicine requiring professional review.
19. Patient with verified allergy information.
20. Patient with complete medical records.
21. Patient receiving multiple prescribed medicines.
22. Patient with a medicine dosage requiring verification.
23. Patient with an unknown medical condition.
24. Patient with an existing medicine interaction.
25. Patient with missing prescription details.
26. Patient with a confirmed safe-use condition.
27. Patient with a dangerous medicine combination.
28. Patient with an age-related restriction.
29. Patient with conflicting medical information.
30. Patient with complete information and no identified contraindication.

Knowledge Base Size Comparison

The system should be evaluated using three different Knowledge Base sizes.

Knowledge Base size	**Number of rules**
Small	40
Medium	400
Large	2,000

The following performance measurements should be recorded:

- Query latency: Time required to answer a query.
- Inference steps: Number of logical operations performed.
- Memory usage: Amount of memory required.
- Explanation length: Number of steps or reasons provided in the decision.

The system should also be compared with a baseline reasoning method to understand the benefits of the selected approach.

Actual performance values must be obtained by running the Python implementation.

Stage 5 – Cross-Defence

1. Student A Explains Experimental Evidence

The experimental evidence helps us understand how the system performs with different Knowledge Base sizes.

When the number of rules increases from 40 to 400 and then to 2,000, the system may require more time and inference steps.

Indexing and goal-based reasoning help reduce unnecessary operations.

2. Student B Explains Theoretical Complexity

The theoretical complexity depends on the number of rules, facts and relationships involved in a query.

Backward Chaining focuses on the required goal rather than processing every available fact.

Therefore, it can be more efficient when only a small portion of the Knowledge Base is relevant.

Stage 6 – Joint Evaluation

1. Why Is This Problem Suitable for a Logical Agent?

The medication-safety problem is suitable for a logical agent because it involves clear rules, patient information and relationships between medicines and medical conditions.

The system can apply logical reasoning to identify possible risks and explain its decisions.

2. Limitations of the Representation

The system has some limitations:

1. It cannot always handle uncertain medical information.
2. It may produce incomplete results if patient records are missing.
3. It depends on the correctness of the Knowledge Base.
4. It may not identify unknown drug interactions.
5. It cannot replace the knowledge and judgement of qualified healthcare professionals.

3. What Should an AI Designer Check Before Choosing a Knowledge-Based Approach?

An AI designer should consider:

1. The complexity of the problem.

2. The number of rules and facts.
3. The availability and accuracy of information.
4. The type of reasoning required.
5. The processing time and memory requirements.
6. The need for explanations.
7. The safety and reliability of the system.

Contribution Record

Your assignment requires a contribution record at the end. You can use the following table and update it according to the actual work completed by both students.

Task	**Student A – Primary Contribution**	**Student B – Primary Contribution**	**Joint Work**
Task 1	Symbol analysis and inference rules	Model checking and soundness	Wumpus World KB
Task 2	Resolution analysis	Model-checking comparison	Access-control policy
Task 3	FOL vocabulary and axioms	Query investigation	Eligibility system
Task 4	Experimental analysis	Unification and correctness	Lifted inference
Task 5	CNF conversion and proof	Experimental comparison	Resolution prover
Task 6	Forward Chaining	Backward Chaining	Organisation hierarchy
Task 7	Theoretical validation	Experimental validation	Hospital safety system